

Machado User Manual

machado provides users with a framework to store, search, and visualize biological data.

It is powered by Django and contains tools to interact with a Chado database.

Contents

1. Installation
 2. Data Loading
 - Ontologies
 - Taxonomy
 - Organism
 - Publication
 - FASTA
 - GFF
 - Feature Additional Info
 - BLAST
 - InterProScan
 - OrthoMCL
 - RNA-seq
 - Coexpression
 - VCF
 3. Visualization
 - Index and Search
 - Web Server
 - Customization
 - JBrowse
 4. Diagrams
 5. Models
-

Technical Stack Documentation

For developer-oriented documentation covering the database schema, REST API, loaders internals, and CI/CD, see the `techstack/` directory.

1. Installation

machado is a Django framework for Chado.

Prerequisites

PostgreSQL 16

Install PostgreSQL and create a database and user for loading the Chado schema.
As the `postgres` user run:

```
psql
CREATE USER username WITH ENCRYPTED PASSWORD 'password';
CREATE DATABASE yourdatabase WITH OWNER username;
ALTER USER username CREATEDB;
```

Don't forget to configure the PostgreSQL server to allow regular users to connect (`pg_hba.conf`).

Linux dependencies

Be sure to have the following packages installed:

```
sudo apt install zlib1g-dev libbz2-dev liblzma-dev python3-dev
```

Python 3.12+

We strongly recommend creating a new virtualenv for your project:

```
mkdir /var/www/MYGENOME
cd /var/www/MYGENOME
python3 -m venv .venv
source .venv/bin/activate
```

Install machado

```
pip install machado
```

Or, to install from the latest source:

```
pip install git+https://github.com/lmb-embrapa/machado.git
```

Create a project

The `machado-startproject` command creates a ready-to-use Django project with pre-configured settings, URL routing, and WSGI/ASGI entry points:

```
machado-startproject .
```

This generates the following files in the current directory:

```
.
|-- .env                # Your configuration (auto-generated SECRET_KEY)
|-- .env.example        # Reference with all available settings
|-- cache/              # Rendered search pages (see Page cache below)
|-- manage.py
```

```

+-- machadoproject/
  |-- __init__.py
  |-- settings.py
  |-- urls.py
  |-- wsgi.py
+-- asgi.py

```

Configure the environment

Edit the `.env` file and set your PostgreSQL connection string:

```

SECRET_KEY=<auto-generated>
DATABASE_URL=postgres://username:password@localhost:5432/yourdatabase

```

The `DATABASE_URL` format follows the `dj-database-url` convention. Replace `username`, `password`, and `yourdatabase` with the values you created in the Prerequisites step.

See `.env.example` for the full list of optional settings including Elasticsearch, JBrowse, and API configuration.

Page cache

The search page (`/find/`) and the data summary (`/data/`) are cached on disk and cleared automatically — when you run `rebuild_search_index`, and when you change an organism’s visibility from the permissions panel. `machado-startproject` created a `cache/` directory for this, which is all a development install needs.

For production, the directory has to be writable by **both** the web server and whoever runs `rebuild_search_index` — a directory inside the project tree usually is not. Put it outside and give both access:

```

sudo install -d -o www-data -g www-data -m 2775 /var/cache/machado
sudo usermod -aG www-data $USER

```

Then set it in `.env`:

```

CACHE_DIR=/var/cache/machado

```

If the directory is not writable the site still works — pages are simply rebuilt on every request — and `python manage.py check` reports it as `machado.W001`.

Two other settings are available, both optional: `CACHE_MAX_ENTRIES` (default 1000000; note that omitting it entirely would mean Django’s own default of 300) and `CACHE_TIMEOUT` (default 3600, applying only to the JBrowse API views, not to cached pages).

Cached pages have no expiry of their own, so only the two events above clear them. If you change the database by some other route — editing rows directly,

say — the cached pages keep the old numbers until one of those events happens. Running `rebuild_search_index` is the way to force it.

Migrate and run

Apply the Chado schema migrations:

```
python manage.py migrate
```

Just ignore warnings about unapplied migrations.

Collect the static files:

```
python manage.py collectstatic --no-input
```

Run the tests to verify the installation:

```
python manage.py test machado
```

Start the development server:

```
python manage.py runserver
```

Open `http://localhost:8000/` in your browser.

References

- http://gmod.org/wiki/Chado_Django_HOWTO

2. Data Loading

The Chado schema heavily relies on **Ontologies** to integrate different datasets. Therefore ontologies must be the first data loaded.

Inside the `extras` directory, there's a file named `sample.tar.gz` that might be helpful. It contains a few small FASTA and GFF files, together with a README file with examples of command lines.

Loading order

The recommended loading order is:

1. Ontologies — **required**, must be loaded first
2. Taxonomy — optional, but useful for multi-organism instances
3. Organism — required if taxonomy was not loaded
4. Publication — optional, allows linking DOIs to data
5. FASTA — reference genome sequences
6. GFF — gene models / genome annotations
7. Feature Additional Info — annotations, sequences, publications, cross-references
8. BLAST — similarity search results

9. InterProScan — protein domain analysis
10. OrthoMCL — orthology groups
11. RNA-seq — expression data
12. Coexpression — coexpression pairs and clusters
13. VCF — variant data

Tip: Most `load_*` commands accept `--cpu` to increase the number of threads and speed up loading.

Loading Ontologies

The ontologies are required and must be loaded first.

Relations Ontology

RO is an ontology of relations for use with biological ontologies.

- **URL:** <https://github.com/oborel/obo-relations>
- **File:** `ro.obo`

```
python manage.py load_relations_ontology --file ro.obo
```

Sequence Ontology

Collection of SO Ontologies.

- **URL:** <https://github.com/The-Sequence-Ontology/SO-Ontologies>
- **File:** `so.obo`

```
python manage.py load_sequence_ontology --file so.obo
```

Gene Ontology

Source ontology files for the Gene Ontology.

- **URL:** <http://current.geneontology.org/ontology/>
- **File:** `go.obo`

```
python manage.py load_gene_ontology --file go.obo
```

- Loading the Gene Ontology can be faster if you increase the number of threads (`--cpu`).
- After loading, the following records will be created in the `Cv` table: `gene_ontology`, `external`, `molecular_function`, `cellular_component`, and `biological_process`.

Remove Ontology

If, for any reason, you need to remove an ontology, use the command `remove_ontology`. Most data files you'll load depend on the ontologies

(e.g. FASTA, GFF, BLAST). You should **never** delete an ontology after having data files loaded.

```
python manage.py remove_ontology --help
```

- This command requires the name of the ontology (`Cv.name`).
- There are dependencies between the Gene Ontology records, therefore you need to delete the entry **external** first.

Loading Taxonomy

Every data file you'll load (e.g. FASTA, GFF, BLAST) must belong to an organism. These are instructions to load the NCBI taxonomy, that contains most organisms you'll need.

This step is optional. If you decide to skip this step, you should insert the organisms individually using the command `insert_organism`.

NCBI Taxonomy

Contains the names and phylogenetic lineages of more than 160,000 organisms that have molecular data in the NCBI databases.

- **URL:** <https://www.ncbi.nlm.nih.gov/taxonomy>
- **File:** <ftp://ftp.ncbi.nlm.nih.gov/pub/taxonomy/taxdump.tar.gz>

After unpacking the file you'll have the required files.

```
python manage.py load_organism --file names.dmp --name DB:NCBI_taxonomy
```

```
python manage.py load_phylotree --file nodes.dmp --name 'NCBI taxonomy tree' --organismdb 'I
```

- Loading these files can be faster if you increase the number of threads (`--cpu`).
- It will take a long time anyway (hours).

Remove Taxonomy

If, for any reason, you need to remove a taxonomy, use the commands `remove_phylotree` and `remove_organisms`. Most data files you'll load depend on the organism database (e.g. FASTA, GFF, BLAST). **If you delete an organism database, every data file you loaded will be deleted on cascade.**

```
python manage.py remove_phylotree --help
```

```
python manage.py remove_organisms --help
```

- These commands require the names of the databases (`Db.name`).

Inserting a New Organism

Insert Organism

Every data file you'll load (e.g. FASTA, GFF, BLAST) must belong to an organism. These are instructions to insert a new organism to the database **if you did NOT** load the NCBI taxonomy or if the organism you're working with is not included.

```
python manage.py insert_organism --genus 'Organismus' --species 'ineditus'
```

- There are optional fields you can provide. Please take a look at the command help (`--help`).

Remove Organism

If, for any reason, you need to remove an organism, use the command `remove_organism`. Most data files you'll load depend on the organism record (e.g. FASTA, GFF, BLAST). **If you delete an organism, every data file you loaded that depends on it will be deleted on cascade.**

```
python manage.py remove_organism --help
```

- These commands require the following info: `Organism.genus` and `Organism.species`.

Loading Publication Files

Load Publication

```
python manage.py load_publication --file reference.bib
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

Remove Publication

If, for any reason, you need to remove a publication, use the command `remove_publication`. **If you delete a publication, every record that depends on it will be deleted on cascade, with the exception of the Dbxref field that contains the DOI accession.**

```
python manage.py remove_publication --help
```

Loading FASTA Files

This command is mostly used to load the reference genome. The reference sequences are exclusively used to feed JBrowse.

If the reference sequences are really long (>200 Mbp), there may be memory issues during the loading process and JBrowse may take too long to render the tracks. To avoid that, it's possible to use the parameter `--nosequence` and configure JBrowse to get the reference data from a FASTA file.

Load FASTA

```
python manage.py load_fasta --file organism_chrs.fa --soterm chromosome --organism 'Arabidopsis thaliana'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_fasta --help
```

Parameter	Description
<code>--file *</code>	FASTA file
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--soterm *</code>	SO Sequence Ontology Term (e.g. chromosome, assembly)
<code>--description</code>	Description
<code>--url</code>	URL
<code>--doi</code>	DOI of a reference stored using <code>load_publication</code> (e.g. 10.1111/s12122-012-1313-4)
<code>--nosequence</code>	Don't load the sequences
<code>--cpu</code>	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a FASTA file, use the command `remove_organism`. Most data files you'll load depend on the organism record (e.g. FASTA, GFF, BLAST). **If you delete an organism, every data file you loaded that depends on it will be deleted on cascade.**

```
python manage.py remove_organism --help
```

- These commands require the following info: `Organism.genus` and `Organism.species`.

Loading GFF Files

The first column of a GFF file is the reference sequence ID. Usually, in order to load a GFF file, it's required to have a reference FASTA file loaded. But some GFF files already have the reference features such as chromosome or scaffold. In this case, there are two options:

- Load the GFF directly, without a reference FASTA file
- Load the FASTA file and then load the GFF using the parameter `--ignore` to not load the reference features

The GFF file must be indexed using tabix.

Load GFF

```
python manage.py load_gff --file organism_genes_sorted.gff3.gz --organism 'Arabidopsis thaliana'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_gff --help
```

Parameter	Description
<code>--file *</code>	GFF3 genome file indexed with tabix (see http://www.htslib.org/doc/tabix.html)
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--ignore</code>	List of feature types to ignore (e.g. chromosome scaffold)
<code>--doi</code>	DOI of a reference stored using <code>load_publication</code> (e.g. 10.1111/s12122-012-1313-4)
<code>--qtl</code>	Set this flag to handle GFF files from QTLDB
<code>--cpu</code>	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a GFF dataset, use the command `remove_file`. **If you delete a file, every record that depends on it will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name (`Dbxrefprop.value`).

Loading Feature Additional Info

Please notice the features must be loaded in advance. If the annotation or sequence was loaded previously, it will be replaced.

Load Annotation

```
python manage.py load_feature_annotation --file feature_annotation.tab --soterm polypeptide
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_annotation --help
```

Parameter	Description
<code>--file *</code>	Two-column tab separated file (<code>feature.accession<TAB>annotation text</code>)
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--soterm *</code>	SO Sequence Ontology Term (e.g. mRNA, polypeptide)
<code>--cvterm *</code>	cvterm.name from cv feature_property (e.g. display, note, product, alias, ontology_term, annotation)
<code>--doi</code>	DOI of a reference stored using <code>load_publication</code> (e.g. 10.1111/s12122-012-1313-4)
<code>--cpu</code>	Number of threads

* required fields

Remove Annotation

If, for any reason, you need to remove a feature annotation, use the command `remove_feature_annotation`.

```
python manage.py remove_feature_annotation --help
```

Load Sequence

```
python manage.py load_feature_sequence --organism 'Arabidopsis thaliana' --file Athaliana_t...
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_sequence --help
```

Parameter	Description
<code>--file *</code>	FASTA file
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--soterm *</code>	SO Sequence Ontology Term (e.g. chromosome, assembly, mRNA, polypeptide)
<code>--cpu</code>	Number of threads

* required fields

Remove Sequence

If, for any reason, you need to remove a feature sequence, use the command `load_feature_sequence` itself and provide a FASTA file with no sequence. For example:

```
>chr1
```

```
>chr2
```

Load Publication

```
python manage.py load_feature_publication --organism 'Arabidopsis thaliana' --file feature_p
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_publication --help
```

Parameter	Description
<code>--file *</code>	Two-column tab separated file (<code>feature.accession<TAB>DOI</code>)
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--cpu</code>	Number of threads

* required fields

Remove Publication

If, for any reason, you need to remove a feature publication attribution, use the command `remove_publication`.

```
python manage.py remove_publication --help
```

Load DBxRef

```
python manage.py load_feature_dbxrefs --organism 'Arabidopsis thaliana' --file feature_dbxre
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_feature_dbxrefs --help
```

Parameter	Description
<code>--file *</code>	Two-column tab separated file (<code>feature.accession<TAB>db:dbxref</code>)

Parameter	Description
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--soterm *</code>	SO Sequence Ontology Term (e.g. chromosome, assembly, mRNA, polypeptide)
<code>--cpu</code>	Number of threads

* required fields

Loading BLAST Results

In order to load a BLAST XML result file, both the query and the subject records must be previously stored (see below). The current version was tested for loading BLAST analysis on **proteins**.

Load BLAST Subject Records

In case you did a BLAST against a multispecies protein database, like NCBI's nr or Uniprot's trembl or swissprot, you need to load all subject matches before loading the result itself:

```
python manage.py load_similarity_matches --file blast_result.xml --format blast-xml
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

Load BLAST

If all queries and subjects are already loaded you can run the following command to load a BLAST XML result:

```
python manage.py load_similarity --file blast_result.xml --format blast-xml \
    --so_query polypeptide --so_subject protein_match \
    --program diamond --programversion 0.9.24 \
    --organism_query 'Oryza sativa' --organism_subject 'multispecies multispecies'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_similarity --help
```

Parameter	Description
<code>--file *</code>	BLAST XML file
<code>--format *</code>	blast-xml
<code>--so_query *</code>	Query Sequence Ontology term (e.g. assembly, mRNA, polypeptide)
<code>--so_subject *</code>	Subject Sequence Ontology term (e.g. protein_match)

Parameter	Description
<code>--organism_query</code>	Query's organism name (e.g. <i>Oryza sativa</i>). Cannot be
*	'multispecies'.
<code>--organism_subject</code>	Subject's organism name. Put multispecies multispecies
*	if using a multispecies database.
<code>--program *</code>	Program
<code>--programversion</code>	Program version
*	
<code>--name</code>	Name
<code>--description</code>	Description
<code>--algorithm</code>	Algorithm
<code>--cpu</code>	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a BLAST result set, use the command `remove_analysis`.

```
python manage.py remove_analysis --help
```

- This command requires the analysis name (`Analysis.sourcename`).

Loading InterProScan Results

In order to load an InterProScan results file, both the query and the subject records must be previously stored. The current version was tested for loading InterProScan analysis on **proteins**.

Load InterProScan Subject Records

```
python manage.py load_similarity_matches --file interproscan_result.xml --format interproscan
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

Load InterProScan Similarity

```
python manage.py load_similarity --file interproscan_result.xml --format interproscan-xml \
    --so_query polypeptide --so_subject protein_match \
    --program interproscan --programversion 5 \
    --organism_query 'Oryza sativa' --organism_subject 'multispecies multispecies'
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_similarity --help
```

Parameter	Description
<code>--file *</code>	InterProScan XML file
<code>--format *</code>	<code>interproscan-xml</code>
<code>--so_query *</code>	Query Sequence Ontology term (<code>polypeptide</code>)
<code>--so_subject *</code>	Subject Sequence Ontology term (<code>protein_match</code>)
<code>--organism_query *</code>	Query's organism name (e.g. <i>Oryza sativa</i>). Cannot be 'multispecies'.
<code>--organism_subject *</code>	Subject's organism name. Put <code>multispecies multispecies</code> if using a multispecies database.
<code>--program *</code>	Program
<code>--programversion *</code>	Program version
<code>--name</code>	Name
<code>--description</code>	Description
<code>--algorithm</code>	Algorithm
<code>--cpu</code>	Number of threads

* required fields

Remove File

If, for any reason, you need to remove an InterProScan result set, use the command `remove_analysis`.

```
python manage.py remove_analysis --help
```

- This command requires the analysis name (`Analysis.sourcename`).

Loading OrthoMCL Results

Load OrthoMCL

```
python manage.py load_orthomcl --file groups.txt
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_orthomcl --help
```

Parameter	Description
<code>--file *</code>	Output result from OrthoMCL software
<code>--cpu</code>	Number of threads

* required fields

Remove Orthology

If, for any reason, you need to remove orthology relationships, use the command `remove_feature_annotation`.

```
python manage.py remove_feature_annotation --help
```

Parameter	Description
<code>--cvterm *</code>	orthologous group
<code>--organism</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)

* required fields

Loading RNA-seq Data

Load RNA-seq Information

Before inserting RNA-seq count tables, it is needed to input information about the experiments and samples from which the data was generated. In machado we focus on the GEO/NCBI database as a source for RNA-seq experiments information and data.

From the GEO/NCBI database you get identifiers for different *series* (e.g. GSE85653) that describe the studies/projects as a whole. From the GEO series you get identifiers for biosamples (or, in Chado lingo, biomaterials — e.g. GSM2280286). From the GEO biosamples you get identifiers for RNA-seq experiments (or assays), usually from the SRA database (e.g. SRR4033018). SRA identifiers link to the raw data you can analyse.

In machado, it is necessary to input a `.csv` file with information for all SRA data files regarding RNA-seq assays that will be loaded.

This file must have the following fields in each line:

"Organism", "GEO series", "GEO sample", "SRA identifier", "Assay description", "Treatment description"

A sample line:

`Oryza sativa,GSE85653,GSM2280286,SRR4033018,Heat leaf rep1,Heat stress,Leaf,May-30-2018`

To load such a file:

```
python manage.py load_rnaseq_info --file file.csv --biomaterialdb GEO --assaydb SRA
```

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_rnaseq_info --help
```

Parameter	Description
<code>--file *</code>	<code>.csv</code> file
<code>--biomaterialdb</code>	Biomaterial database info (e.g. GEO)
<code>--assaydb</code>	Assay database info (e.g. SRA)
<code>--cpu</code>	Number of threads

* required field

Remove RNA-seq Information

If, for any reason, you need to remove RNA-seq information, use the command `remove_file --name`. **Every relation from the filename (e.g. `file.csv`) will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name used as input to load RNA-seq information.

Load RNA-seq Data

To load expression count tables for RNA-seq data, a tabular file should be loaded. This file can contain data from several RNA-seq experiments (assays) per column. The file should have the following header:

```
Gene identifier      SRA Identifier 1      SRA Identifier 2      ...      SRA Identifier n
```

Example header for a file with two assays/experiments:

```
gene      SRR5167848.htseq      SRR2302912.htseq
```

The body of the table is composed of the gene identifier followed by the counts for each gene, in each experiment:

```
AT2G44195.1.TAIR10      0.0      0.6936967934559419
```

Note that the count fields can have floats or integers, depending on the normalization used (usually TPM, FPKM or raw counts).

The gene identifier is supposed to already be loaded as a feature, usually from the organism's genome annotation GFF file. We used the output of the LSTrAP program as standard format for this file.

```
python manage.py load_rnaseq_data --file file.tab --organism 'Oryza sativa' --programversion
```

- As default the program name is LSTrAP but can be changed with `--program`.
- The data is by default taken as normalized (TPM, FPKM, etc.) but can be changed with `--norm`.

- Loading this file can be faster if you increase the number of threads (`--cpu`).

```
python manage.py load_rnaseq_data --help
```

Parameter	Description
<code>--file</code>	Tabular text file with gene counts per line
<code>--organism</code>	Scientific name (e.g. <i>Oryza sativa</i>)
<code>--programversion</code>	Version of the software (e.g. 1.3)
<code>--name</code>	Optional name
<code>--description</code>	Optional description
<code>--algorithm</code>	Optional algorithm description
<code>--assaydb</code>	Optional assay database info (e.g. SRA)
<code>--timeexecuted</code>	Optional date software was run. Mandatory format: e.g. Oct-16-2016
<code>--program</code>	Optional name of the software (default: LSTrAP)
<code>--norm</code>	Optional. Normalized data: 1 = yes (TPM, FPKM, etc.); 0 = no (raw counts); default 1

Remove RNA-seq Data

If, for any reason, you need to remove RNA-seq data, use the command `remove_file --name`. **Every relation from the filename (e.g. file.tab) will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name used as input to load RNA-seq data.

Loading Coexpression Analyses

Load Coexpression Correlated Pairs

Coexpression analyses usually generate correlation statistics regarding gene sets in a pairwise manner. For example, LSTrAP can generate a table with the Pearson correlation coefficient for every pair of genes in an RNA-seq experiment.

To load such a table into machado, the file must be headless and tab separated, with the two first columns containing the correlated pair of IDs for the genes/features and the third column containing the correlation coefficient. In the case of the LSTrAP software output, the coefficient is subtracted by 0.7 for normalization.

A sample from such a table:

AT2G44195.1.TAIR10	AT1G30080.1.TAIR10	0.18189286870895194
AT2G44195.1.TAIR10	AT5G24750.1.TAIR10	0.1715779378273995

Note: The feature pairs from columns 1 and 2 need to be loaded previously.

To load such a table:

```
python manage.py load_coexpression_pairs --organism 'Arabidopsis thaliana' --file pcc.mcl.txt
```

- Loading this file can be faster if you increase the number of threads (--cpu).

```
python manage.py load_coexpression_pairs --help
```

Parameter	Description
--file *	Coexpression pairs file (e.g. pcc.mcl.txt)
--organism *	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
--soterm	Sequence ontology term (default: mRNA)
--cpu	Number of threads

* example output file from LSTrAP software

Remove Coexpression Pairs

If, for any reason, you need to remove coexpression pair analyses, pass the filename used to load the analyses to the `remove_relationship` command. **Every coexpression relation from the file will be deleted on cascade.**

```
python manage.py remove_relationship --help
```

Parameter	Description
--file *	Coexpression file (e.g. mcl.clusters.txt)

Load Coexpression Clusters

Another type of coexpression analysis involves clustering features using their correlation values. LSTrAP does this using the `mcl` software. The input file must be headless and tab separated, with each line representing one cluster and each column representing one gene/feature from that cluster. The first column should represent the cluster name in the format `<cluster_name>:.`

Three-cluster sample (the third line is an orphaned cluster with only one individual — orphaned clusters are discarded):

```
ath_1:    AT3G18715.1.TAIR10    AT3G08790.1.TAIR10    AT5G42230.1.TAIR10
ath_2:    AT1G27040.1.TAIR10    AT1G71692.1.TAIR10
ath_3:    AT5G24750.1.TAIR10
```

Note: The genes/features from each column must be loaded previously.

To load such a file:

```
python manage.py load_coexpression_clusters --file mcl.clusters.txt --organism 'Arabidopsis
```

- Loading this file can be faster if you increase the number of threads (--cpu).

```
python manage.py load_coexpression_clusters --help
```

Parameter	Description
--file *	Clusters file (e.g. <code>mcl.clusters.txt</code>)
--organism	Scientific name (e.g. <i>Arabidopsis thaliana</i>)
--soterm	Sequence ontology term (default: mRNA)
--cpu	Number of threads

Remove Coexpression Clusters

If, for any reason, you need to remove coexpression cluster analyses, pass the controlled vocabulary term `coexpression group` and the organism scientific name to the command `remove_feature_annotation`. **Every coexpression group relation from that organism will be deleted on cascade.**

```
python manage.py remove_feature_annotation --help
```

Parameter	Description
--cvterm *	<code>coexpression group</code>
--organism	Scientific name (e.g. <i>Oryza sativa</i>)

Loading VCF Files

The first column of a VCF file is the reference sequence ID. In order to load a VCF file, it's required to have a reference FASTA file loaded.

The VCF file must be indexed using tabix.

Load VCF

```
python manage.py load_vcf --file organism_snv_sorted.vcf.gz --organism 'Arabidopsis thaliana
```

- Loading this file can be faster if you increase the number of threads (--cpu).

```
python manage.py load_vcf --help
```

Parameter	Description
<code>--file *</code>	VCF genome file indexed with tabix (see http://www.htslib.org/doc/tabix.html)
<code>--organism *</code>	Species name (e.g. <i>Homo sapiens</i> , <i>Mus musculus</i>)
<code>--doi</code>	DOI of a reference stored using <code>load_publication</code> (e.g. 10.1111/s12122-012-1313-4)
<code>--cpu</code>	Number of threads

* required fields

Remove File

If, for any reason, you need to remove a VCF dataset, use the command `remove_file`. **If you delete a file, every record that depends on it will be deleted on cascade.**

```
python manage.py remove_file --help
```

- This command requires the file name (`Dbxrefprop.value`).

Visualization

The machado web interface requires the installation and configuration of third party software.

- Index and Search — PostgreSQL full-text search
- Web Server — Development server and Apache WSGI deployment
- JBrowse — Genome browser integration (optional)

Index and Search

PostgreSQL Full-Text Search

Machado uses the native **full-text search capabilities of PostgreSQL** to provide fast, reliable search and faceted navigation without requiring any external search services. All search indexes are stored within the main database.

`MACHADO_VALID_TYPES=gene,mRNA,polypeptide`

If `MACHADO_VALID_TYPES` is not explicitly set in the settings, the default is `['gene', 'mRNA', 'polypeptide']`.

Indexing the Data

The search system relies on a denormalized “materialized” index table to guarantee fast responses. After loading data into the Chado database, you must build the search index to make the new records searchable:

```
python manage.py rebuild_search_index
```

Note: It is necessary to run `rebuild_search_index` whenever additional data is loaded into the database or when you wish to refresh the search facets.

Rebuilding also clears the page cache, which is what makes newly loaded data visible on the search page and on the data summary. See Page cache for its one requirement, a writable cache directory.

Rebuilding the index walks the features in chunks: for each chunk it issues a fixed, small number of queries for all the related data the chunk needs, then bulk-inserts that chunk’s rows. `--batch-size` sets the size of that chunk, so it controls both how many rows are inserted at once *and* how many features each prefetch query covers — every IN list in the prefetch grows with it.

The default of 2000 is a deliberate compromise: larger chunks mean fewer round-trips but bigger IN lists and more memory held per chunk, so raise it only if you have measured a benefit on your data.

Smaller chunks: more round-trips, less memory per chunk
`python manage.py rebuild_search_index --batch-size 500`

Resuming an interrupted rebuild

A full rebuild over millions of features takes hours. If it is interrupted, re-run with `--resume` to continue from where it stopped:

```
python manage.py rebuild_search_index --resume
```

`--resume` skips features already present in the index and does not clear it. It assumes `MACHADO_VALID_TYPES` has not changed since the interrupted run; if you changed that setting, use `--restart` to rebuild from scratch.

In the web interface, the “Resume interrupted run” checkbox on the Rebuild Search Index form is the on/off switch for this same behaviour: leaving it unchecked is the `--restart` behaviour (the default), and checking it is `--resume`.

--resume is additive only. It indexes only features whose `feature_id` is above the highest `feature_id` already in the index. That makes it the right tool for continuing an interrupted run, and the wrong tool for everything else:

- It will **not** refresh data attached to features that are already indexed. Loaders such as `load_similarity`, `load_feature_annotation`, `load_orthomcl` and either

`load_coexpression_*` command attach new data to *pre-existing* features, whose `feature_ids` are below the watermark; `--resume` skips them and their index rows stay stale. Use `--restart` after any such load.

- It will **not** remove index rows for features that have since been deleted or marked obsolete. Only `--restart` clears those.

The web form’s warning text covers only the first, more common failure mode; this page is the canonical, complete reference for both.

Flag	Effect
<i>(none)</i>	Clear the index and rebuild everything (default)
<code>--restart</code>	Same as the default, stated explicitly
<code>--resume</code>	Continue an interrupted run (additive only, see above)
<code>--batch-size N</code>	Features per chunk (default 2000)
<code>--max-features N</code>	Stop after N features, for benchmarking

`search_vector` is a PostgreSQL generated column, so no separate tsvector update step runs after indexing: it is maintained by the database itself and must not be assigned directly.

Search Features

The PostgreSQL search backend supports advanced search queries natively using “websearch” syntax:

- `gene kinase` — searches for documents containing both words
- `"receptor kinase"` — searches for the exact phrase
- `kinase -receptor` — searches for “kinase” but excludes documents containing “receptor”
- `kinase OR receptor` — searches for either word

All exported data (TSV or FASTA) from the search page is unlimited; PostgreSQL handles exporting the full result set regardless of its size.

Web Server

Django manage runserver

Check if your server is working — test the machado server:

```
python manage.py runserver
```

Now, open your browser and go to `http://127.0.0.1:8000`.

Use CTRL+C to stop the web server.

Django Apache WSGI

Before starting, make sure you have Apache installed in your system. In Ubuntu 20.04:

```
sudo apt install apache2
```

In order to have Apache2 hosting the Django applications, it's necessary to use WSGI. By doing this it won't be necessary to run the runserver command anymore — Apache will take care of this process. Install the following package:

```
sudo apt install libapache2-mod-wsgi-py3
```

Now configure Apache to use the WSGI module. Here is the configuration file (/etc/apache2/sites-available/MYGENOME.conf):

```
<Directory "/var/www/MYGENOME/machadoproject">
    <Files "wsgi.py">
        Require all granted
    </Files>
</Directory>
```

```
Alias /MYGENOME/static/ /var/www/MYGENOME/staticfiles/
```

```
<Directory "/var/www/MYGENOME/staticfiles">
    Require all granted
</Directory>
```

```
WSGIDaemonProcess machadoproject python-home=/var/www/MYGENOME/.venv python-path=/var/www/MYGENOME/machadoproject
WSGIProcessGroup machadoproject
WSGIScriptAlias /MYGENOME /var/www/MYGENOME/machadoproject/wsgi.py
```

- In this example the whole project is in /var/www/MYGENOME, but it's not required to be there.
- This directory and sub-directories must have 755 permissions.
- Ensure the python-home path matches your virtual environment (.venv).

There must be a symlink of your config file in the sites-enabled directory:

```
sudo ln -s /etc/apache2/sites-available/MYGENOME.conf /etc/apache2/sites-enabled/MYGENOME.conf
```

In the .env file, ensure the following variables are set for production:

```
DEBUG=False
ALLOWED_HOSTS=*
CSRF_TRUSTED_ORIGINS=https://yourdomain.com
TIME_ZONE=America/Sao_Paulo
```

```
MACHADO_VALID_TYPES=gene,mRNA,polypeptide
```

Set TIME_ZONE to the server's actual timezone (defaults to UTC if unset). Django

applies it at process startup, so it also determines what wall-clock time is recorded for timestamps such as the loader’s command history — a mismatch here makes those timestamps look off relative to the server’s local clock.

If you are running more than one machado instance on the same machine — for example, several instances mounted at different Apache subpaths on the same domain — set `URL_PREFIX` in each instance’s `.env` to that instance’s mount path (e.g. `URL_PREFIX=/demo`). Without it, session/CSRF cookies and the browser-stored accent color are not scoped per instance, so signing in or picking a color in one instance can interfere with the others:

```
URL_PREFIX=/demo
```

Now, run `collectstatic` to gather the static files from all libraries to the `staticfiles` directory:

```
python manage.py collectstatic
```

It’s necessary to restart the Apache2 service every time there are modifications on configuration files or source code updates:

```
sudo systemctl restart apache2.service
```

Now, open your browser and go to your configured domain (e.g., `http://yourdomain.com/MYGENOME`).

Customization Guide

Machado Genomics provides built-in options to customize the landing page and add Release Notes. This makes it simple for users to adapt each installation to specific biological species or data portals without editing the source code.

1. Landing Page Customization

You can customize almost all titles, descriptions, icons, and texts on the homepage by setting environment variables in your `.env` file (or configuring them directly in your system/container environment).

How to Hide Optional Sections

If your installation does not require all feature cards or all “How it works” steps, you can **hide** any card or step by setting its corresponding `_TITLE` variable to an empty string (e.g. `MACHADO_FEATURE3_TITLE=""` or `MACHADO_STEP3_TITLE=""`).

The same pattern applies to several whole sections, which are hidden automatically when their controlling setting is empty:

- The “Key Features & Capabilities” heading and subtitle are hidden when `MACHADO_FEATURES_TITLE` is set to an empty string.

- The “How It Works” heading and subtitle are hidden when MACHADO_HOWITWORKS_TITLE is set to an empty string — the three step cards below it (MACHADO_STEP1_TITLE etc.) are unaffected and keep their own individual hide rule.
- The Acknowledgements section is hidden unless MACHADO_ACKNOWLEDGEMENTS_TEXT is set — it is **empty by default**, so this section does not appear at all until you configure it.
- The footer’s extra text block is hidden unless MACHADO_FOOTER_TEXT is set — it is also **empty by default**.
- The live stats panel (Organisms / Genomic Features / Ingestion Formats) is hidden by setting MACHADO_SHOW_STATS=False. This one is a boolean rather than an empty title, because the panel has no title to blank out; turning it off also skips the two database counts behind it, which are unfiltered scans of the organism and feature tables.

Settings Reference

Below is a complete reference of the 29 customizable settings, their corresponding environment variables, and their default values:

Environment Variable	Description	Default Value
Site-wide Settings		
MACHADO_ACCENT_COLOR	Accent color theme for the UI (accent-picker swatches in the navbar). Valid values: steel , teal , sage , amber , graphite , ruby , indigo , emerald , coral , plum .	"steel"
MACHADO_SITE_TITLE	Browser tab title and navbar brand.	"Machado Genomics"
MACHADO_SITE_DESCRIPTION	HTML meta description tag for search engine optimization (SEO).	"Machado Genomics - biological data management"
Hero Section		
MACHADO_HERO_TITLE	Main heading in the landing page hero banner.	"Genomics Data Management Portal"

Environment Variable	Description	Default Value
MACHADO_HERO_SUBTITLE	Paragraph text shown below the main heading in the hero banner.	"Explore, search, and visualize genomics sequences, annotations, and publications stored in the Chado database."
Key Features Section		
MACHADO_FEATURES_TITLE	Heading above the feature cards. Setting this to an empty string hides the whole heading and subtitle.	"Key Features & Capabilities"
MACHADO_FEATURES_SUBTITLE	Subtitle text shown below the Key Features heading.	"A comprehensive ecosystem designed for biological database curation and research."
Feature Card 1		
MACHADO_FEATURE1_TITLE	Title of the first feature card.	"Multi-Format Data Loaders"
MACHADO_FEATURE1_TEXT	Description of the first feature.	"Ingest data seamlessly from standard bioinformatics formats including FASTA, GFF3, OBO, BibTeX, BLAST, InterProScan, and OrthoMCL directly into the Chado relational schema."
MACHADO_FEATURE1_ICON	FontAwesome 5 CSS class name for the card icon.	"fas fa-file-import"
Feature Card 2		
MACHADO_FEATURE2_TITLE	Title of the second feature card.	"PostgreSQL Faceted Search"

Environment Variable	Description	Default Value
MACHADO_FEATURE2_TEXT	Description of the second feature.	"Execute complex queries powered by PostgreSQL full-text search. Filter features by organism, sequence ontology terms, orthology, coexpression groups, and related publications."
MACHADO_FEATURE2_ICON	FontAwesome 5 CSS class name for the card icon.	"fas fa-search"
Feature Card 3		
MACHADO_FEATURE3_TITLE	Title of the third feature card.	"Genome Browser Integration"
MACHADO_FEATURE3_TEXT	Description of the third feature.	"Interactive visual analysis of features. Machado Genomics API delivers data directly to the embedded JBrowse genome browser for sequence and annotation alignments."
MACHADO_FEATURE3_ICON	FontAwesome 5 CSS class name for the card icon.	"fas fa-align-left"
Live Stats Panel		
MACHADO_SHOW_STATS	Whether to show the stats panel below the feature cards. Set to False to hide all three cards and skip the two counts that back them.	True
How It Works Heading		

Environment Variable	Description	Default Value
MACHADO_HOWITWORKS_TITLE	Heading above the “How It Works” step cards. Setting this to an empty string hides the whole heading and subtitle, but not the step cards below it.	"How Machado Genomics Operates"
MACHADO_HOWITWORKS_SUBTITLE	Subtitle text shown below the How It Works heading.	"From raw genomic files to interactive database search and visualization."
Step 1 (How It Works)		
MACHADO_STEP1_TITLE	Title for step 1 of the platform overview.	"Load Data"
MACHADO_STEP1_TEXT	Explanation of how data loading works.	"Administrators run commands or use data tools to load genomic files into the database."
Step 2 (How It Works)		
MACHADO_STEP2_TITLE	Title for step 2 of the platform overview.	"Index & Query"
MACHADO_STEP2_TEXT	Explanation of how searching works.	"PostgreSQL full-text index updates automatically, enabling fast, multi-faceted searches across millions of features."
Step 3 (How It Works)		
MACHADO_STEP3_TITLE	Title for step 3 of the platform overview.	"Discover"
MACHADO_STEP3_TEXT	Explanation of how discovery/viewing works.	"Users inspect features, view analysis results, download bulk data, and browse via JBrowse."

Environment Variable	Description	Default Value
Acknowledgements		
Section (optional)		
MACHADO_ACKNOWLEDGEMENTS_TITLE	Heading for the Acknowledgements section.	"Acknowledgements"
MACHADO_ACKNOWLEDGEMENTS_TEXT	Body text for the Acknowledgements section. This section is hidden from the page entirely unless this setting is non-empty; it is empty by default.	""
Footer		
MACHADO_FOOTER_COPYRIGHT	Copyright line shown in the footer.	"© 2026 Embrapa. All rights reserved."
MACHADO_FOOTER_TEXT	Optional extra text block shown above the footer's copyright/attribution row. Hidden entirely unless set; empty by default.	""

The footer's "Powered by Machado Genomics" line is a hardcoded literal and is **not** configurable via any setting — it deliberately does not reflect MACHADO_SITE_TITLE or any other setting, so renaming a site instance never rewords it.

Note also that MACHADO_HOWITWORKS_TITLE is a plain static setting, not linked to MACHADO_SITE_TITLE — overriding MACHADO_SITE_TITLE alone no longer changes the "How It Works" heading; set MACHADO_HOWITWORKS_TITLE explicitly if you want it to match.

Upgrading an existing project: the eight settings MACHADO_FEATURES_TITLE, MACHADO_FEATURES_SUBTITLE, MACHADO_HOWITWORKS_TITLE, MACHADO_HOWITWORKS_SUBTITLE, MACHADO_ACKNOWLEDGEMENTS_TITLE, MACHADO_ACKNOWLEDGEMENTS_TEXT, MACHADO_FOOTER_COPYRIGHT, and MACHADO_FOOTER_TEXT were added to the project template after some existing projects were generated (and MACHADO_ACCENT_COLOR may be missing too, if your project predates that setting). If your machadoproject/settings.py predates one of these settings, setting the corresponding environment variable in your .env is a **silent no-op**: nothing reads it into a Django setting, so the page just falls back to the built-in default, with no error or warning.

To use these settings in an existing project, add the matching `env(...)` lines to your own `machadoproject/settings.py` by hand, copying them from the “Landing page customization” section of `machado/project_template/machadoproject/settings.py` in the machado package. Regenerating the project from the template is **not** a safe shortcut — using `--overwrite` would clobber any customizations you have already made to that file.

2. Release Notes

A “Release Notes” section is dynamically displayed at the bottom of the landing page. It is structured as an accordion list using Bootstrap 4 where only the latest version note is expanded by default, and all previous versions are collapsed.

Display Logic

- **Automatic Toggle:** If there is no `release_notes.json` file in your project root, the Release Notes section will be **completely hidden** from the landing page.
- **Accordion Style:** When a file is found, it renders as a card deck where clicking on a version header toggles its description. The first item in the list is always expanded on page load.
- **Caching:** For optimal performance, the Release Notes file is read and parsed **once at server startup**. If you modify `release_notes.json`, you must restart your web server (e.g. Apache, Gunicorn, or Django runserver) for the changes to take effect.

Configuration Format

To define release notes, create a file named `release_notes.json` in your project’s base directory (the directory containing `manage.py` and your `.env` file). The file must contain a JSON array of objects, each representing a release.

Three keys are read from each object:

- **version** — written **without** a leading `v`. The landing page prepends one, so `"1.2.0"` renders as `v1.2.0` and `"v1.2.0"` would render as `vv1.2.0`.
- **date** — shown next to the version, as given. Any format you like.
- **description** — rendered as a single block of plain text. Markdown is not interpreted and line breaks are not preserved, so write one paragraph of prose rather than a bulleted list.

Entries appear in the order given, and the first is expanded on page load — so put the newest release first.

Here is an example format of a `release_notes.json` file:

```
[
  {
    "version": "1.2.0",
    "date": "2026-06-12",
    "description": "Added customizable landing page settings and a brand new Release Notes a
  },
  {
    "version": "1.1.0",
    "date": "2026-04-05",
    "description": "Improved PostgreSQL search performance and added full-text index trigger
  },
  {
    "version": "1.0.0",
    "date": "2026-01-10",
    "description": "Initial stable release of the Machado Genomics data portal framework wit
  }
]
```

JBrowse (optional)

The JBrowse software renders genomic tracks from Chado using the machado API.

Installing JBrowse

Before installing JBrowse you should probably have BioPerl installed in your system (tested in Ubuntu 20.04):

```
sudo apt install bioperl
```

Then install JBrowse following the official instructions.

In Ubuntu 20.04, install some prerequisites:

```
sudo apt install build-essential zlib1g-dev curl
```

Also Node.js is needed:

```
curl -sL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Finally, proceed with JBrowse installation:

```
wget https://github.com/GMOD/jbrowse/releases/download/1.16.12-release/JBrowse-1.16.12.zip
unzip JBrowse-1.16.12.zip
sudo mv JBrowse-1.16.12 /var/www/html/jbrowse
cd /var/www/html
sudo chown `whoami` jbrowse
```

```
cd jbrowse
./setup.sh # don't do sudo ./setup.sh
```

Configuring JBrowse

The machado source contains the directory `extras`. Copy the file `extras/trackList.json.sample` to the JBrowse data directory, inside a subdirectory with the name of the organism (e.g. `/var/www/html/jbrowse/data/Arabidopsis thaliana`) and rename it to `trackList.json`. This directory structure will enable machado to embed JBrowse in its pages.

```
mkdir -p "/var/www/html/jbrowse/data/Arabidopsis thaliana"
cp extras/trackList.json.sample "/var/www/html/jbrowse/data/Arabidopsis thaliana/trackList.json"
```

Edit the file `trackList.json` to set the **organism** name you have loaded to the database.

In case you have WSGI apache module configured and running, change the `baseUrl` variable in `trackList.json` to refer to the proper address:

```
"baseUrl": "http://localhost/MYGENOME/api/jbrowse"
```

- Repeat the steps above for as many other organisms as you may have loaded to the database.
- Remember to restart the Apache server after the modifications.

machado API

In case you don't have the WSGI module installed under Apache (and did not change the `baseUrl` variable in `trackList.json`), start the machado API framework:

```
python manage.py runserver
```

Once the server is running, go to your browser and open your JBrowse instance (e.g. `http://localhost/jbrowse/?data=data/Arabidopsis%20thaliana`).

machado Settings

The `settings.py` file should contain these variables in order to have the JBrowse visualization embedded in the feature page. Don't forget to restart Apache to make the settings changes up to date.

```
MACHADO_JBROWSE_URL = 'http://localhost/jbrowse'
MACHADO_JBROWSE_TRACKS = 'ref_seq,gene,transcripts,CDS,SNV'
MACHADO_JBROWSE_OFFSET = 1200
```

- `MACHADO_JBROWSE_URL` — the base URL to the JBrowse installation. The URL must contain the protocol (i.e. `http` or `https`).
- `MACHADO_JBROWSE_TRACKS` — the name of the tracks to be displayed (`ref_seq,gene,transcripts,CDS,SNV` if not set).

- `MACHADO_JBROWSE_OFFSET` — the number of bp upstream and downstream of the feature (1000 if not set).

Use Reference from FASTA File (optional)

If the reference sequences are really long (>200 Mbp), there may be memory issues during the loading process and JBrowse may take too long to render the tracks. To avoid this, follow instructions to create and use an indexed FASTA file as source for the reference sequences — see the JBrowse tutorial.

Follow the steps below.

Put the genome's assembly FASTA file into your JBrowse organism's `data/seq` directory (for example: `/var/www/html/jbrowse/data/Glycine max/data/seq/Gmax.fa`), change to this directory, and run:

```
samtools faidx Gmax.fa
```

A `Gmax.fa.fai` indexed FASTA file will be created.

Now, modify your default `trackList.json` file. You need to make two modifications — first replace the `refSeqs` entry line (probably the second line of the file) with:

```
"refSeqs" : "data/seq/Gmax.fa.fai",
```

And then change the reference sequence track code chunk:

```
{
  "category" : "1. Reference sequence",
  "faiUrlTemplate" : "data/seq/Gmax.fa.fai",
  "key" : "Reference sequence",
  "label" : "ref_seq",
  "seqType" : "dna",
  "storeClass" : "JBrowse/Store/SeqFeature/IndexedFasta",
  "type" : "SequenceTrack",
  "urlTemplate" : "data/seq/Gmax.fa",
  "useAsRefSeqStore" : 1
}
```

- The code above should replace all code from the “1. Reference sequence” category track code chunk.
- Make sure `urlTemplate` points to the FASTA file path, not the `.fai` indexed one.

Now restart the Apache daemon for changes to take effect:

```
systemctl restart apache2
```

Diagrams

AnalysisFeature

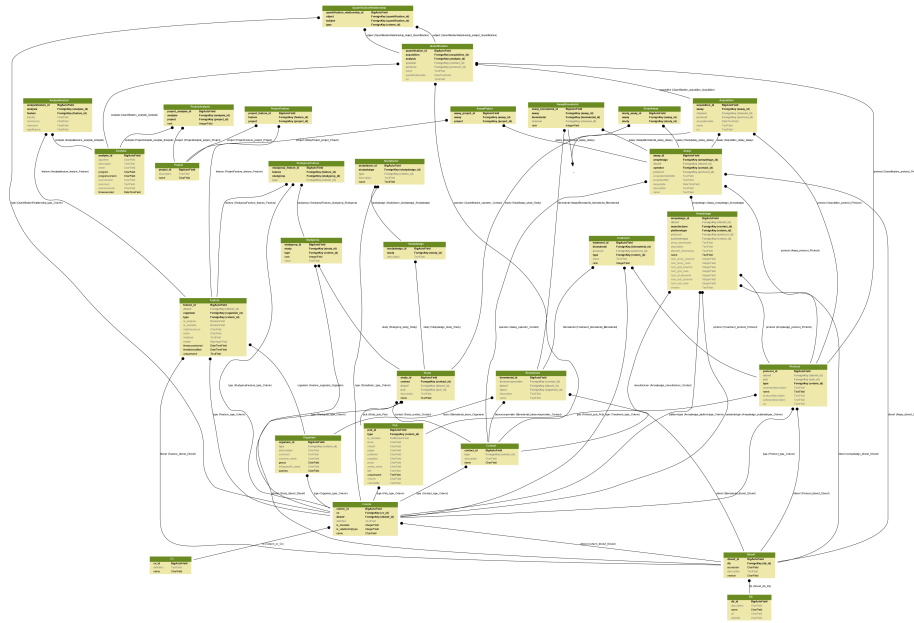


Figure 1: AnalysisFeature diagram

Feature

Ontology

Publication

Sequence

Similarity

Taxonomy

Note: Instructions to generate the diagrams: https://django-extensions.readthedocs.io/en/latest/graph_models.html

Example: `python manage.py graph_models --pygraphviz -a -I Analysis,Analysisfeature,Feature -o analysis_feature.png`

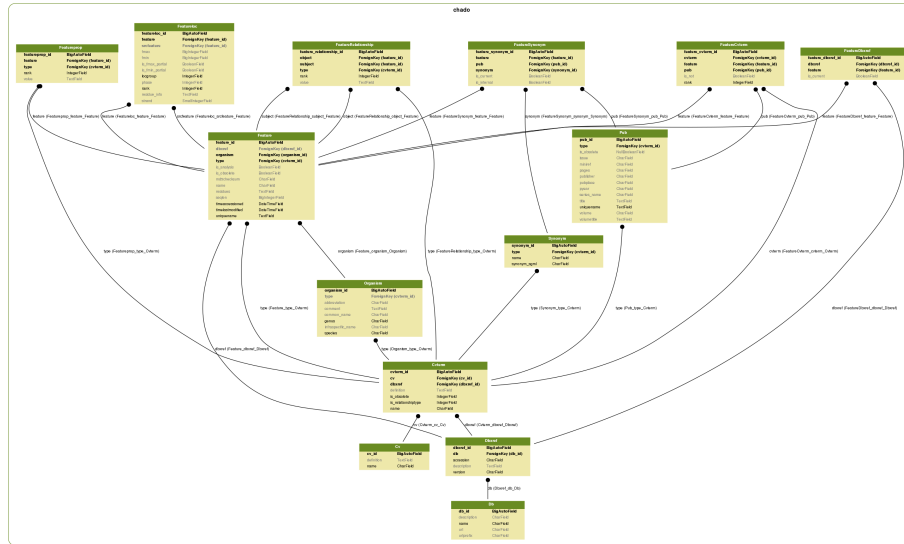


Figure 2: Feature diagram

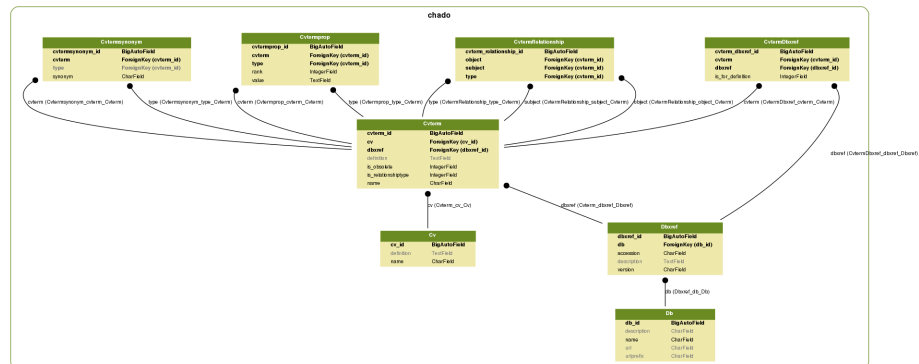


Figure 3: Ontology diagram

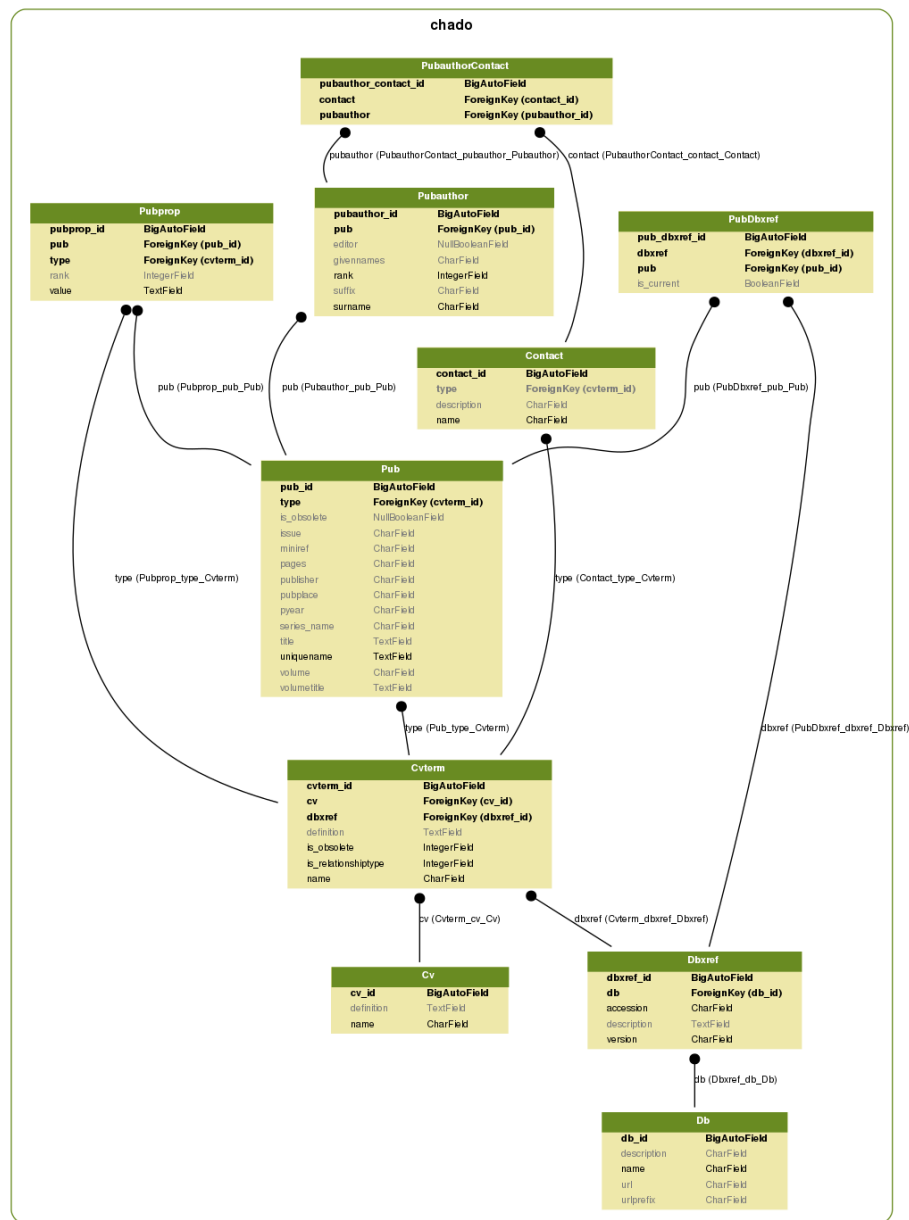


Figure 4: Publication diagram

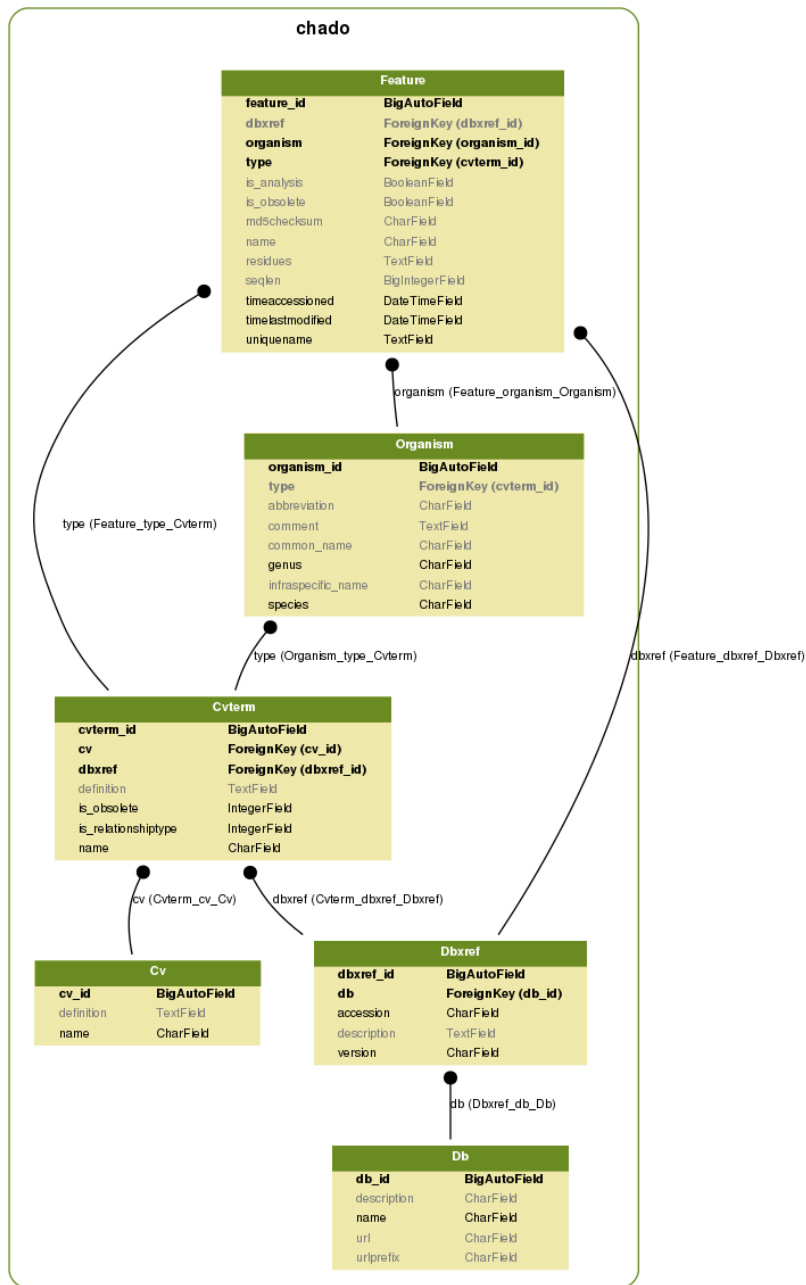


Figure 5: Sequence diagram

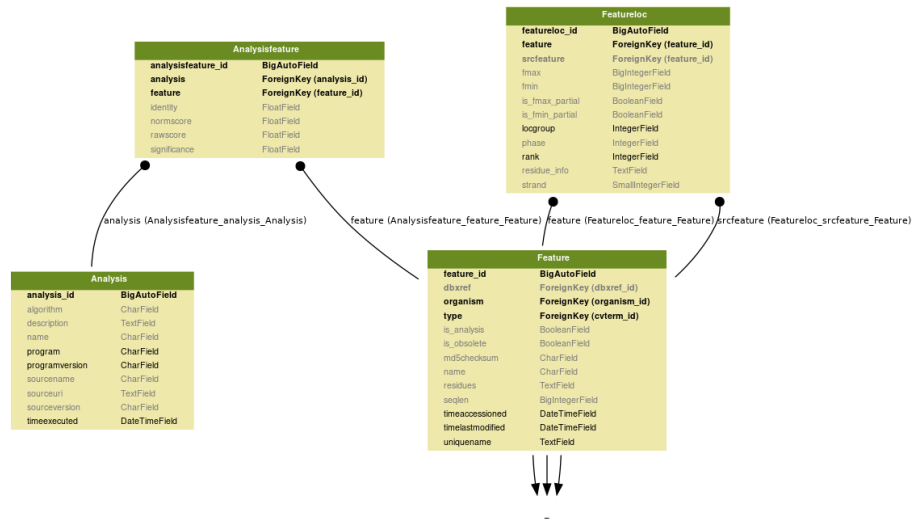


Figure 6: Similarity diagram

Models

This document describes how the Django `models.py` file for the Chado schema was created. You **don't need** to create it again since machado already contains a copy of this file.

Prerequisite

The list below contains the software and versions required by machado.

PostgreSQL 9.5

Install PostgreSQL and create a database and user for loading the Chado schema. As the `postgres` user run:

```
psql
CREATE USER username WITH ENCRYPTED PASSWORD 'password';
CREATE DATABASE yourdatabase WITH OWNER username;
```

Chado 1.31

Download the Chado schema, unpack the file and load the `chado-1.31/schemas/1.31/default_schema.sql` to the database:

```
psql -h localhost -U username -W -d yourdatabase < chado-1.31/schemas/1.31/default_schema.sql
```


Python 3.5.2

We strongly recommend creating a new virtualenv for your project:

```
virtualenv -p /usr/bin/python3 MYGENOME
cd MYGENOME
source bin/activate
```

machado

Just grab the code using Git and install it:

```
git clone https://github.com/lmb-embrapa/machado.git src/machado
python src/machado/setup.py install
```

The Django Project

Inside the MYGENOME directory, create a Django project with the following command:

```
django-admin startproject WEBPROJECT
cd WEBPROJECT
```

Then, configure the WEBPROJECT/settings.py file to connect to your Chado database:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql_psycopg2',
        'NAME': 'yourdatabase',
        'USER': 'username',
        'PASSWORD': 'password',
        'HOST': 'localhost',
        'PORT': '',
    },
}
```

The Model

Django has a command to generate a models file:

```
python manage.py inspectdb > unsortedmodels.py
```

This will create a raw models.py with a model for each table and view in the specified Postgres database. This file needs to be fixed as each foreign key relation should have a unique name in Django to support reverse relationships. The following Python code will create these unique names. The code rewrites the models and also generates an admin.py file:

```
fixChadoModel.py --input unsortedmodels.py
```

The resulting files, `models.py` and `admin.py`, are ready.

References

- http://gmod.org/wiki/Chado_Django_HOWTO